PS2 (Written)

Graded

Student

Rob Redmon

Total Points

10 / 10 pts

Question 1

3a

2 / 2 pts

✔ **+ 2 pts** Correct answer + explanation

+ 1 pt Incorrect answer with/without justification

Question 2

3b

2 / 2 pts

✔ **+ 2 pts** Correct answer + explanation

+ 1 pt Incorrect answer with some correct justification

– 2 pts No answer

Question 3

4a

4 / 4 pts

✔ **– 0 pts** Correct

– 1 pt No rationale for why the parameter was chosen

– 1 pt No plot

– 2 pts No discussion based on the plot

– 4 pts No answer

Question 4

4b (Extra Credit)

2 / 2 pts

✔ **– 0 pts** Correct

– 1 pt Incorrect answer

– 2 pts No answer

Question assigned to the following page: 1

This handout includes space for every question that requires a written response. Please feel free to use it to hand write your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

3.a [12 points (Coding)] [4 points (Written)] Train and Analysis

Once you have completed problems 1 and 2, you can train your few shot classification model. You should observe both the support and query losses go down, and the query accuracy go up. Now we will examine how the performance varies for different size problems. Train models for the following values of K and N:
K = 1, N = 2; K = 2, N = 2; K = 1, N = 3; K = 1, N = 4

Example code:

```
python main.py --num_shot K --num_classes N
```

For checking training results and/or taking a screenshot for the writeup, use:

```
tensorboard --logdir runs/
```

You should start with the case K = 1, N = 2 as it can aid you in the implementation and debugging process.

Your model should be able to achieve a query set accuracy of above 90% in this first two scenarios scenario on held-out test tasks, around 80% in the third scenario, and around 70% in the final scenario. We have provided in Figure 3 a unified view of the meta-test query set classification accuracy over training iterations for all above mentioned configurations. Answer the following questions:

(a) How does increasing the number of classes affect learning and performance?

Answer: Increasing the number of classes (N) decreases performance.

Details: Looking at the figure below of the meta task Test accuracy and loss curves, we can easily see that increasing the number of classes (N) decreases accuracy in performance (left figure) on the task of predicting the Query image class labels, and correspondingly increases the losses (right figure).

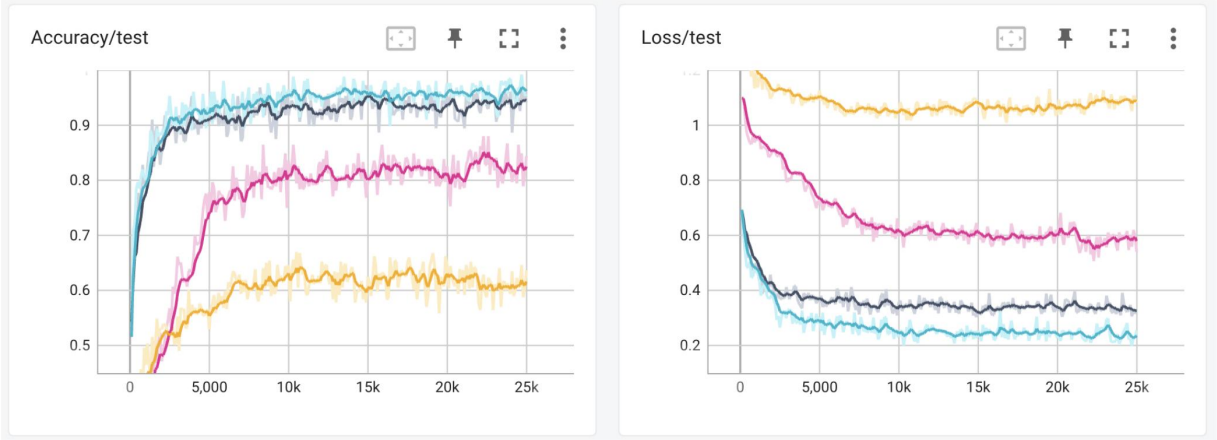


Figure: MANN meta task test accuracy (left) and test loss (right) curves as a function of epoch colored as follows: Black: "K=1, N=2", Blue: "K=2, N=2" Red: "K=1, N=3" Yellow: "K=1, N=4". (Chart settings: Smoothing was set to 75% to easy synoptic interpretation. Ignore outliers in chart scaling = True.)

Question assigned to the following page: 2

3.b How does increasing the number of examples in the support set affect performance?

Answer: Increasing Support examples (K) increases performance.

Details: Looking at the previous figure, specifically the N=2 classes scenarios (Black (K=1) and Blue (K=2) curves), we can see that adding additional Support Images and Labels (i.e. K) does improve accuracy in this particular experiment. So, we should expect that if we increase K further for the N¿2 class cases, perhaps those accuracies (N=3,4) might be increased as well.

Question assigned to the following page: 3

4.a [3 points (Written, Extra Credit)] Experimentation

a. Experiment with one hyperparameter that affects the performance of the model, such as the type of recurrent layer, size of hidden state, learning rate, or number of layers and submit a plot of the loss and accuracy for train and test sets. Provide a brief rationale for why you chose the parameter and what you observed in the caption for the plot.

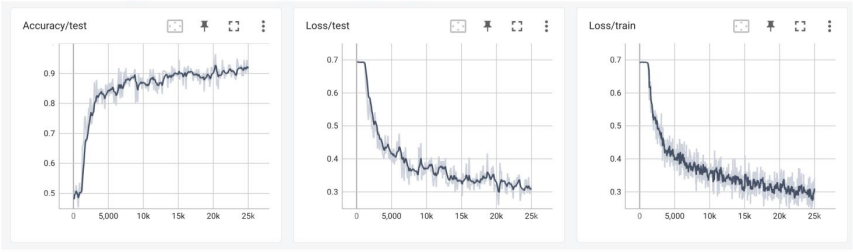
Looking at main.py, the hyperparameters we have easy command line access to manipulate are (including their default values):

```
hidden_dim=128, learning_rate=0.001, train_steps=25000,
```

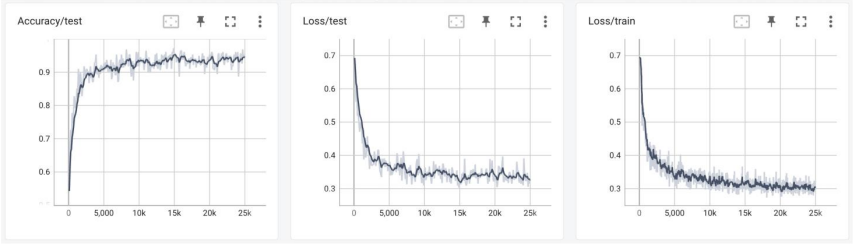
Since we'll try LSTM memory representation in the subsequent exercise 4.b, let's try learning rate here as something easy to experiment with.
Rational for Choice: Learning rate was chosen since it's always important to look at the computational cost per performance to train models (e.g. ecological footprint), and it was easy to experiment with.

Observations: Unsurprisingly, the default learning rate (0.001) is a pretty good choice for balancing iterations needed to achieve greater than 90% accuracy in task testing (middle-left plot). It would be interesting to increase the epochs to 50k and see if the slower learning rate captures more mutual information between the tasks and hence improved Training (right column) and Test (left 2 column) performance as the slowest rate clearly hasn't fully converged.

Slow Learning Rate 0.0001 (default / 10):



Default Learning Rate 0.001 (default):



Fast Learning Rate 0.01 (default*10):

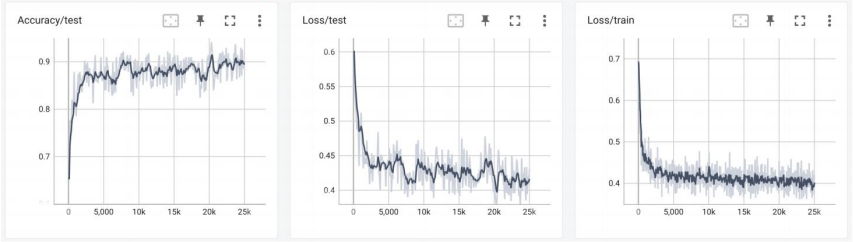


Figure: MANN meta task test accuracy (left), and test loss (middle), and training Loss (right) curves as a function of epoch. The figure rows top to bottom are the learning rates: slowest (top, lr=0.0001, default/10), default (middle, lr=0.001), and fastest (bottom, lr=0.01). Smoothing was set to 75% to easy synoptic interpretation.

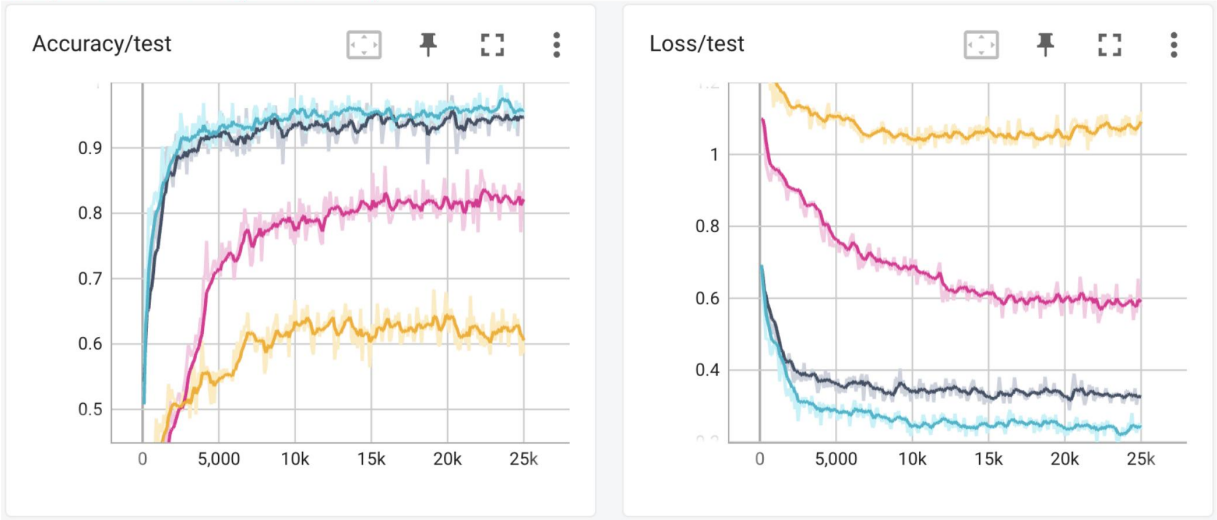
Question assigned to the following page: 3

4.a Experimenting with another change: Adding random noise to the Query labels instead of simply 0's.

Here we replace our zeroing out of the Query labels with labels drawn from a normal distribution ($\mu = 0, \sigma = 1$).

Findings: No performance gain or loss is clearly observable. Perhaps another test to try would be to draw randomly from a permutation of possible integer class labels rather than real values.

Query labels = Normal($\mu = 0, \sigma = 1$) :



Compared to Original (Query labels as 0's):

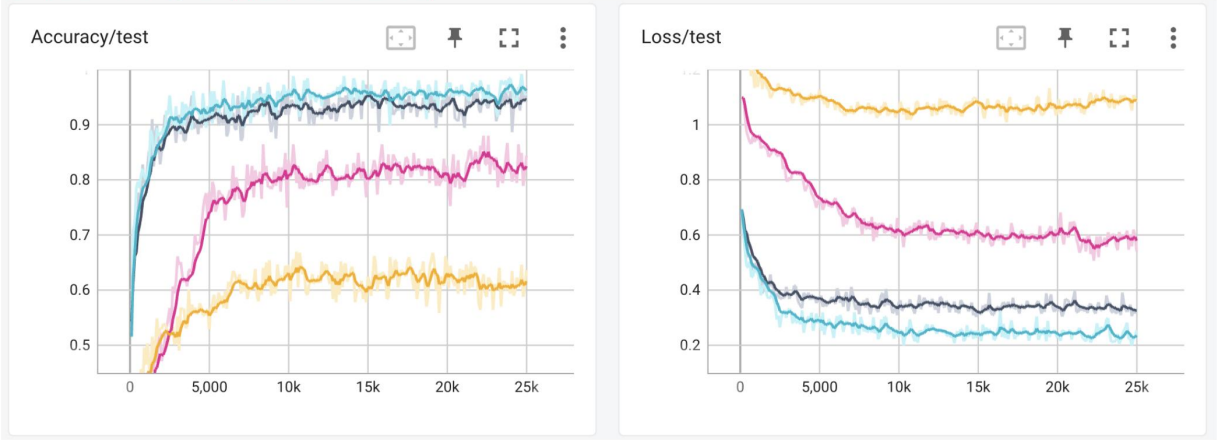


Figure: MANN meta task test accuracy (left) and test loss (right) curves as a function of epoch colored as follows: Black: "K=1, N=2", Blue: "K=2, N=2" Red: "K=1, N=3" Yellow: "K=1, N=4". (Chart settings: Smoothing was set to 75% to easy synoptic interpretation. Ignore outliers in chart scaling = True.)

Question assigned to the following page: 4

4.b [3 points (Written, Extra Credit)] Experimentation

b. In this question we'll explore the effect of memory representation on model performance. We will focus on the K = 1, N = 3 case.

In the previous experiments we used an LSTM model with 128 units. Consider additional memory sizes of 256, and 8.

How does increasing and decreasing the memory capacity influence performance?

Answer: Increasing memory (8 vs. 128(default) vs. 256 LSTM units) increases accuracy. Note that increasing the number of memory units also increases the learning time needed (number of epochs) which is of course as expected, as the number of neural network parameters needing to learn have increased.

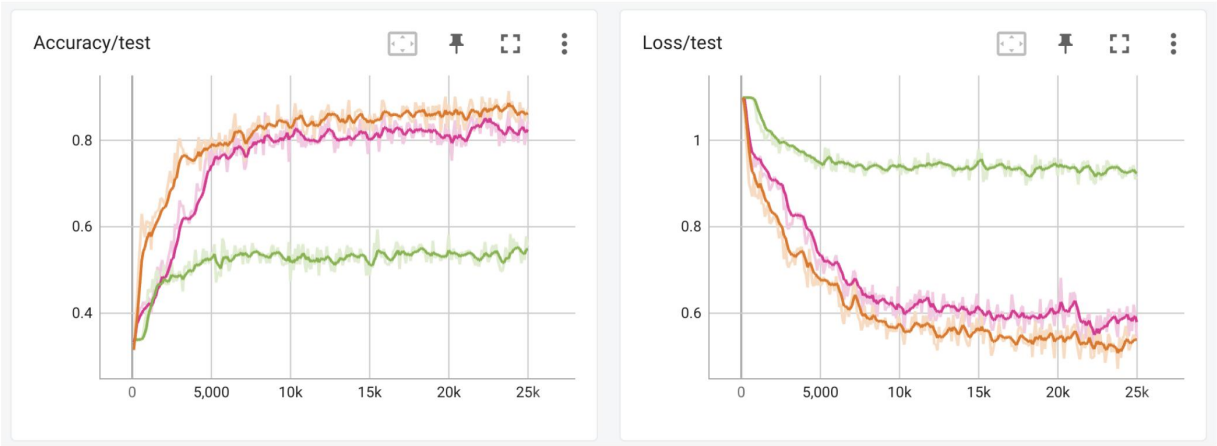


Figure: MANN meta task test accuracy (left) and test loss (right) curves as a function of epoch, for K=1 and N=3 and varying levels of LSTM memory representation (H). Curves are colored as follows: H=256 units (orange), H=128 units (red), H=8 units (green). Smoothing was set to 75% to easy synoptic interpretation. Here "H" refers to the hidden dimension (aka "hidden_dim") in our 2 layer LSTM network.

And as a double check, we can see that our created figure looks as it should, compared to the problem set handout:

